Тестовое задание для Frontend разработчика

Важно: Ссылку на выполненное тестовое необходимо указать в резюме в блок «О себе».

Срок выполнения: 2 дня

Формат сдачи: ссылка на Gitverse

Стек: React.js

Бизнес кейс: Контроль качества данных по целям сотрудников

Проект: Реализовать требования бизнес-заказчика в части расчёта и визуализации процедуры контроля качества данных (далее — ККД). В корпоративной системе управления целями появится модуль ККД: в нём администраторы системы, руководители департаментов и ответственные за целеполагание в бизнес-подразделениях на периодической основе (планируется еженедельно) проверяют качество данных по целям и ключевым результатам.

Проблема: Сотрудники заносят цели и ключевые результаты в корпоративную систему управления целями, и качество этих данных нужно отслеживать. Если цели или ключевые результаты заведены некорректно, процесс целеполагания работает неэффективно, а руководство компании не видит реальной картины по работе над целями, которые само же и ставит.

Как процесс работает сейчас (AS IS)

Процесс ККД сейчас считает технические метрики, без нужной визуализации.

Как процесс должен работать (TO BE)

- Механизм ККД работает по существующим данным целей и ключевых результатов в PostgreSQL.
- Подсчитываемые метрики соответствуют бизнес-требованиям и учитывают каскадирование целей.
- Endpoint-ы в существующих сервисах на Java передают результаты работы модуля ККД на фронтенд.
- В системе управления целями реализована страница модуля ККД: статистика, найденные проблемы и дельта с прошлым прогоном по данным.
- Руководители и администраторы системы проверяют качество данных на периодической основе — планируется еженедельно.

Ключевые сценарии использования (User Stories)

- Как пользователь сервиса, я хочу видеть количественные и качественные метрики по существующему набору данных (цели и ключевые результаты) с учётом каскадирования.
- Как пользователь сервиса, я хочу видеть динамику (дельту) по сравнению с предыдущим набором данных, полученным в прошлый раз.

Тестовое задание

Ссылка на API: https://gitverse.ru/kkk_s/moc_api_kkd

Задача: Реализовать страницу модуля ККД: статистика качества данных, найденные проблемы и дельта с прошлым прогоном. Приложение работает с готовым mock API (инструкция по запуску ниже). Бэкенд писать не нужно. Визуал — по корпоративному UI-киту: справочник токенов и компонентов лежит в

репозитории mock API, файл UI-KIT.md. Там же готовая тема Ant Design — её можно скопировать целиком.

- **Контекст и сводка.** Выбор подразделения (GET /api/meta, units с parent_id) и прогона (GET /api/runs) влияет на всю страницу. Сводка — GET /api/kkd/summary?run_id=&unit_id=: карточки метрик со значением, прошлым значением, дельтой и trend. Направление дельты и оценка «лучше / хуже» — разные вещи: рост числа проблем это плохо, а рост доли корректных целей хорошо; поля higher_is_better и trend приходят из API. Явно показать, с каким прогоном идёт сравнение (compared_to).
- **Найденные проблемы.** GET /api/kkd/issues: таблица с фильтрами по правилу, важности и статусу дельты (new / persisting / resolved) и серверной пагинацией (limit, offset, total). Счётчики new, persisting и resolved из сводки кликабельны и открывают список с этим фильтром.
- **Карточка проблемы.** GET /api/kkd/issues/{id}: описание, фактическое и ожидаемое значение, рекомендация и история по прогонам (history) — в модальном окне или на отдельном экране.
- **Граничные состояния.** RUN-001 — первый прогон: previous и delta равны null, показать «нет данных для сравнения», а не 0. RUN-006 — неуспешный прогон, результаты отдают 409 RUN_FAILED. TX-NEW — подразделение без целей: метрики null. Корректно обрабатывать загрузку (у API есть задержка ответа), ошибку сети и пустой результат фильтра — интерфейс не должен ломаться.

Требования к интерфейсу

В проекте интерфейс принимается по предоставленному Storybook и макетам. В тестовом задании Storybook и макеты не передаются: их заменяет справочник UI-KIT.md из репозитория mock API, по нему и оценивается соответствие визуалу.

Визуал по корпоративному UI-киту: Ant Design 5, палитра, типографика (базовый текст 14/22), радиусы, высоты контролов и отступы — из справочника UI-KIT.md. Приватный npm-пакет кита кандидату недоступен: тему antd нужно собрать из токенов справочника и брать цвета из темы, а не хардкодить hex в компонентах. Шрифт — Inter или любой близкий по метрикам гротеск. Основной сценарий — десктоп от 1280px. Семантическая разметка, базовые принципы доступности (a11y).

Руководитель должен с первого взгляда понять две вещи: стало лучше или хуже по сравнению с прошлым прогоном и что исправлять в первую очередь. Цвет не должен быть единственным способом передать важность проблемы или направление дельты.

Оценивается

- Иерархия информации: что человек видит первым на странице ККД.
- Честность отображения дельты: с чем сравниваем, «лучше / хуже» не путается со знаком числа, отсутствие сравнения не выдаётся за ноль.
- Работа с параметрами: смена подразделения, прогона и фильтров без перезагрузки страницы.
- Поведение в граничных состояниях: неуспешный и первый прогон, пустое подразделение, долгий ответ, ошибка сети.
- Соответствие UI-киту и аккуратность кода: структура папок, отсутствие дублирования запросов, понятные имена.

Технические требования

- Стек: React.js (TypeScript — плюс), Ant Design 5.
- Приложение должно запускаться через docker compose up — предоставить Dockerfile и docker-compose.yml.
- Репозиторий должен содержать README.md с инструкцией запуска (не более 3 команд).

- Адрес API вынести в переменную окружения, не хардкодить в компонентах.
- Не пересчитывать метрики и дельту на фронтенде: все числа берутся из ответов API как есть.
- Не требуется: авторизация, свой бэкенд, SSR. Тесты не обязательны, но будут плюсом.
- **Если решение написано с ИИ-ассистентом** — в репозитории должен быть один артефакт на выбор. Либо папка specs/ со спецификацией, по которой шла работа: requirements.md (что делает решение и критерии приёмки) и design.md (модель данных, API, ключевые решения); если решение менялось по ходу — спецификация обновлена. Либо файл AI-PLAN.md — план работы с ассистентом: какие задачи ему ставились, что принято как есть, что переписано вручную и почему.
- **Если решение написано без ИИ-ассистента** — ничего дополнительно прикладывать не нужно, достаточно одной строки об этом в README.

Mock API: подними локально перед началом работы

```
# нужен Docker
docker compose up --build


# или локально (Python 3.11+)
pip install -r requirements.txt && uvicorn app.main:app --reload
```

Документация эндпоинтов: http://localhost:8000/docs. Справочник правил, уровней важности и метрик — GET /api/meta. Мок отдаёт больше эндпоинтов, чем нужно для задания: обязательны только перечисленные выше, остальные — по желанию.

Данные синтетические и хранятся в памяти. История — 8 еженедельных прогонов ККД, последний в ближайший прошедший понедельник. Перезапуск контейнера или POST /api/reset возвращает демо-данные в исходное состояние.

Демо-данные подобраны под разные состояния интерфейса:

- CIB-REG «Управление региональных продаж» — в последнем прогоне у многих целей сломан каскад, доля корректных целей заметно упала.
- HQ-SME «Департамент малого и среднего бизнеса» — качество данных растёт от прогона к прогону, много исправленных проблем.
- TX-SAL «Управление зарплатных проектов» — в последнем прогоне проблем нет.
- TX-NEW «Отдел платёжных сервисов» — новое подразделение, целей ещё нет.
- RUN-006 завершился с ошибкой, RUN-001 — первый прогон без сравнения.

Мотивация участия в проекте

- Почему тебе интересен этот проект?
- Как ты видишь свою роль в команде, которая создаёт продукт?
- Сколько времени в неделю ты готов(а) уделять проекту и в течение какого периода?